

Codeforces 1500A Pigeonhole

Yes. **Codeforces 1500A — Going Home** is a very good example of the **Pigeonhole Principle**, but the pigeonhole idea is a little hidden.

[Codeforces 1500A — Going Home](#)

The goal is to find **four distinct indices**:

$$a_x + a_y = a_z + a_w$$

where x, y, z, w are all different. 

Small correction: the problem number is **1500A**, not 1501C.

1. First understand the key observation

Suppose we choose **two indices** i, j .

They produce a pair sum:

$$a_i + a_j$$

We want to find **two different pairs** having the same sum.

For example:

```
Array:
1 3 5 7

Pairs:
1+3 = 4
1+5 = 6
1+7 = 8
3+5 = 8 ← collision
3+7 = 10
5+7 = 12
```

We have:

$$1 + 7 = 3 + 5$$

and all four indices are different.

So the problem essentially becomes:

Can we find two disjoint pairs having the same pair-sum?

2. Where does Pigeonhole come in?

This is the important part.

The values satisfy:

$$1 \leq a_i \leq 2.5 \times 10^6$$

Therefore the **maximum possible pair sum** is:

$$2.5 \times 10^6 + 2.5 \times 10^6 = 5 \times 10^6$$

So there are at most roughly:

$$5 \times 10^6$$

different possible pair sums.

These are our **pigeonholes**.

Pigeonholes

```
Sum = 2
Sum = 3
Sum = 4
Sum = 5
...
Sum = 5,000,000
```

Every pair of indices produces one of these sums.

Therefore:

Pairs = pigeons

Possible sums = pigeonholes

If we generate **more than 5 million pairs**, two pairs **must have the same sum**.

That's the pigeonhole principle.

3. But there is a problem

We need **four distinct indices**.

Suppose:

```
a[1] = 5
a[2] = 7
a[3] = 7
```

Then:

$$a_1 + a_2 = 5 + 7 = 12$$

and

$$a_1 + a_3 = 5 + 7 = 12$$

We found equal sums, but the pairs are:

```
(1, 2)
(1, 3)
```

They share index 1.

So this is **not valid**.

Therefore, we cannot blindly rely on pair-sum collision.

4. First handle duplicate values

This is the clever preprocessing.

There are several cases.

Case 1 — Some number occurs 4 times

Suppose:

```
5 5 5 5
```

Then simply:

$$5 + 5 = 5 + 5$$

Using four different indices.

So immediately:

```
YES
```

5. Case 2 — Two different numbers occur twice

Suppose:

```
2 2 7 7
```

Then:

$$2 + 7 = 2 + 7$$

Using:

```
(1, 3)
(2, 4)
```

All indices are different.

So:

YES

6. Case 3 — One number occurs twice

Suppose:

5 5 2 8

We can try pairs involving one 5:

$$5 + 2 = 7$$

$$5 + 8 = 13$$

But if another pair gives the same sum, we can potentially construct the answer.

The standard solution handles these duplicate cases first.

After removing the easy duplicate situations, we can effectively focus on the case where the remaining numbers are **all distinct**.

7. Now the real Pigeonhole argument

Suppose we have sufficiently many **distinct values**.

For every pair:

$$(i, j), \quad i < j$$

calculate:

$$a_i + a_j$$

There are only at most:

$$5,000,000$$

possible sums.

But if we examine more than 5 million pairs, by the **Pigeonhole Principle**, two pairs must have the same sum.

For example, imagine:

Pairs	Sum
(1,2) ----->	100
(1,3) ----->	250
(1,4) ----->	500
...	
(2000,1999) ----->	731
(2010,1998) ----->	731 ← collision

Therefore:

$$a_i + a_j = a_k + a_l$$

8. Why can we guarantee the indices are different?

This is where the duplicate preprocessing matters.

Suppose:

$$a_i + a_j = a_i + a_k$$

Then:

$$a_j = a_k$$

But after handling/restricting duplicates appropriately, this situation cannot create a problematic collision among the remaining distinct values.

So when we find a repeated pair sum, we verify:

```
C++
i != k
i != l
j != k
j != l
```

If all are different:

YES

Otherwise continue.

9. Why don't we use all $n = 200,000$ elements?

Because:

$$\binom{200000}{2}$$

is approximately:

20 billion

which is far too many pairs.

But we **don't need that many**.

The pigeonhole principle tells us that once we have enough pairs to exceed the number of possible sums, a collision is guaranteed.

Since there are about 5 million possible sums, we only need around a few thousand numbers because:

$$\binom{k}{2} > 5,000,000$$

Let's estimate:

$$k \approx 3163$$

because:

$$\binom{3163}{2} \approx 5,000,000$$

So we can work with only a few thousand carefully selected elements.

This is the beautiful part of the problem:

```
n can be 200,000
↓
we don't process all pairs
↓
pair sums have only ~5,000,000 possibilities
↓
Pigeonhole
↓
a few thousand elements are enough
```

10. The complete intuition

Think of the problem like this:

Step 1 — We need

$$a_x + a_y = a_z + a_w$$

with four different indices.

Step 2 — Convert the problem

Instead of thinking about 4 elements, think about **pairs**.

```
(i, j) → a[i] + a[j]
```

Step 3 — Number of possible pair sums

Because:

$$1 \leq a_i \leq 2.5 \times 10^6$$

we have:

$$2 \leq a_i + a_j \leq 5 \times 10^6$$

So there are only about **5 million pigeonholes**.

Step 4 — Generate more than 5 million pairs

If we have more than 5 million pairs:

$$\text{pigeons} > \text{holes}$$

therefore:

two pairs must have the same sum

Step 5 — Ensure four different indices

Handle duplicates first and verify the four indices when a collision occurs.

11. The pattern you should remember

This problem teaches a very useful competitive-programming pattern:

If you can map many objects into a relatively small number of possible states, Pigeonhole Principle can guarantee a collision.

Here:

```
OBJECT
↓
pair of indices (i,j)
↓
STATE
↓
a[i] + a[j]
↓
only ~5 million possible states
```

So:

many pairs + few possible sums $\Rightarrow$ collision

And that collision gives us the desired equal-sum pairs.

The official constraints are $n \leq 200000$ and $a_i \leq 2.5 \cdot 10^6$, which is exactly what makes this counting argument useful. [Codeforces+1](#)

If you want, I can next explain the **exact implementation of 1500A line-by-line**, including **why we keep only certain duplicate elements and why 3163/5000 elements are enough**.

Today 9:34 AM

Sure. Here is the standard **Pigeonhole Principle + pair-sum** solution for **Codeforces 1500A — Going Home**.

```
C++
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<int> a(n);

    for (int &x : a)
        cin >> x;

    // We only need a small number of elements.
    // 5000 elements give us more than enough pairs.
    n = min(n, 5000);

    unordered_map<int, pair<int, int>> mp;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int sum = a[i] + a[j];

            if (mp.count(sum)) {
                auto [x, y] = mp[sum];

                // Check that all four indices are different
                if (x != i && x != j &&
                    y != i && y != j) {
                    cout << "YES\n";
                    cout << x + 1 << " "
                        << y + 1 << " "
                        << i + 1 << " "
                        << j + 1 << "\n";
                    return 0;
                }
            }
        }
    }
    else {
```

```

        mp[sum] = {i, j};
    }
}
cout << "NO\n";
return 0;
}

```

The important part

This:

```

C++
unordered_map<int, pair<int, int>> mp;

```

stores:

```
sum → pair of indices
```

For example:

```
mp[10] = {2, 5}
```

means:

$$a_2 + a_5 = 10$$

Then if we later find:

```
a[3] + a[7] = 10
```

we have:

$$a_2 + a_5 = a_3 + a_7$$

and if all indices are different, we found the answer.

Why only 5000 elements?

We have at most about:

$$5 \times 10^6$$

possible pair sums.

With 5000 elements:

$$\binom{5000}{2} = 12,497,500$$

pairs.

So:

$$12.5 \text{ million pairs} > 5 \text{ million possible sums}$$

By **Pigeonhole Principle**, a collision is guaranteed.

That's the core trick of the problem:

```

5000 elements
↓
~12.5 million pairs
↓
only ~5 million possible sums
↓
Pigeonhole Principle
↓
same sum must occur
↓
check 4 distinct indices
↓
YES

```

Complexity: approximately $O(5000^2)$, which is about **12.5 million pair checks per test case**, with $O(5 \cdot 10^6)$ worst-case hash-map storage.

Yes — the key idea is very simple.

Pigeonhole with remainders

Suppose we divide numbers by some number m .

Every number has a remainder between:

$$0, 1, 2, \dots, m - 1$$

So there are only **m possible remainders**.

If we have **more than m numbers**, by the **Pigeonhole Principle**, at least two numbers must have the **same remainder**.

Suppose two numbers are:

$$a, b$$

and they have the same remainder:

$$a \bmod m = b \bmod m$$

That means we can write:

$$a = qm + r$$

and

$$b = pm + r$$

Notice that the **same r** appears.

Now subtract:

$$a - b = (qm + r) - (pm + r)$$

The remainders cancel:

$$a - b = qm - pm$$

$$a - b = (q - p)m$$

Therefore:

$$m \mid (a - b)$$

So **the difference is not necessarily 0**.

That's the important point.

Example

Take $m = 5$.

$$\begin{aligned} 7 \% 5 &= 2 \\ 12 \% 5 &= 2 \end{aligned}$$

Same remainder.

But:

$$12 - 7 = 5$$

The difference isn't 0.

Instead:

$$12 - 7 = 5$$

which is **divisible by 5**.

Another example:

$$\begin{aligned} 17 \% 5 &= 2 \\ 32 \% 5 &= 2 \end{aligned}$$

Then:

$$32 - 17 = 15$$

Again:

$$15 \bmod 5 = 0$$

Remember this rule

Same remainder $\rightarrow$ difference is divisible by the divisor.

Not:

✗ Same remainder $\rightarrow$ difference is 0

But:

✓ Same remainder $\rightarrow$ difference is a multiple of the divisor.

For example:

$$17 \equiv 32 \pmod{5}$$

means:

$$17 - 32 = -15$$

and:

$$-15 \equiv 0 \pmod{5}$$

This is one of the most common applications of the **Pigeonhole Principle + modular arithmetic** in competitive programming.

Absolutely. The **Pigeonhole Principle (PHP)** appears in many competitive-programming problems, often disguised as something else.

The fundamental idea is always:

More objects than possible categories $\rightarrow$ at least two objects fall into the same category.

Then we use the fact that those two objects having the same category gives us some useful property.

Here are the most important applications you should recognize.

1. Finding duplicate elements

Suppose we have:

1 2 3 4 2

There are 5 numbers, but possible values are only:

1 2 3 4

So:

Objects $\rightarrow$ numbers
Pigeonholes $\rightarrow$ possible values

Since:

$$5 > 4$$

at least two numbers must be the same.

Therefore:

duplicate exists

General pattern

If you have:

$$n + 1$$

objects but only:

$$n$$

possible values/categories, then a duplicate is guaranteed.

2. Same remainder $\rightarrow$ difference divisible by m

This is the one you just asked about.

Suppose we have numbers:

7, 12, 17, 22

and divide by 5.

Possible remainders:

0 1 2 3 4

Only **5 pigeonholes**.

If we have 6 numbers, two must have the same remainder.

Suppose:

$$a \bmod 5 = b \bmod 5$$

Then:

$$a - b$$

is divisible by 5.

Example

$$17 \bmod 5 = 2$$

$$32 \bmod 5 = 2$$

Therefore:

$$32 - 17 = 15$$

and:

$$15 \bmod 5 = 0$$

So:

$$\text{same remainder} \Rightarrow m \mid (a - b)$$

This is extremely useful.

3. Subarray sum divisible by K

This is one of the **most important competitive-programming applications**.

Suppose:

```
a = [2, 3, 1, 6]
k = 5
```

We want to know whether there is a subarray whose sum is divisible by 5.

Step 1: Prefix sums

Calculate:

```
prefix:
2
5
6
12
```

Now take remainders modulo 5:

```
2 % 5 = 2
5 % 5 = 0
6 % 5 = 1
12 % 5 = 2
```

So:

2, 0, 1, 2

We have remainder 2 twice.

Those two prefix sums are:

and:

$$12$$

Their difference:

$$12 - 2 = 10$$

is divisible by 5.

But what does the difference of two prefix sums represent?

$$\text{prefix}[j] - \text{prefix}[i]$$

is exactly the sum of the subarray between them.

Therefore that subarray has a sum divisible by 5.

The Pigeonhole idea

There are only:

$$k$$

possible remainders:

$$0, 1, 2, \dots, k - 1$$

If we have more prefix sums than these categories, two prefix sums must have the same remainder.

Therefore:

$$\text{same prefix remainder} \Rightarrow \text{subarray sum divisible by } k$$

This is a **very important pattern to memorize**.

4. Two numbers whose difference is divisible by K

Suppose you are given n numbers and asked:

Find two numbers whose difference is divisible by k .

You don't actually need to compare every pair.

Just calculate:

$$a_i \bmod k$$

For example:

```
numbers:
7 12 18 23 29
k = 5
```

Remainders:

```
7 → 2
12 → 2
18 → 3
23 → 3
29 → 4
```

We have:

```
2 → 7, 12
3 → 18, 23
```

Since:

$$7 \bmod 5 = 12 \bmod 5$$

we know:

$$12 - 7 = 5$$

which is divisible by 5.

So:

same remainder
↓
difference
↓
divisible by k

5. Birthday Problem

A classic real-life application.

Suppose there are:

$$366$$

possible birthdays if we include leap day.

If there are:

$$367$$

people, then at least two **must** have the same birthday.

Why?

People → pigeons
Possible days → pigeonholes

We have:

$$367 > 366$$

Therefore:

two people must share a birthday

Notice something important:

This does **not** mean that with 50 people a duplicate birthday is guaranteed.

There are 50 people and 366 possible birthdays:

$$50 < 366$$

So PHP doesn't guarantee a collision.

6. Same first/last digit

Suppose you have 101 numbers.

You classify every number according to its **last digit**.

Possible categories:

0 1 2 3 4 5 6 7 8 9

Only 10 categories.

But we have 101 numbers.

Therefore, some last digit occurs many times.

In fact, PHP gives us:

$$\left\lceil \frac{101}{10} \right\rceil = 11$$

So at least **11 numbers** have the same last digit.

This general form is very useful:

At least one category contains $\left\lceil \frac{N}{K} \right\rceil$ objects

where:

- N = objects
- K = categories

7. Same parity

There are only **2 categories**:

```
Even
Odd
```

Suppose we have 3 numbers.

```
2 7 10
```

Categories:

```
Even → 2, 10
Odd  → 7
```

Since:

$$3 > 2$$

at least two numbers must have the same parity.

Therefore, among any 3 integers:

at least two have the same parity

This can immediately tell us something about their sum/difference.

For example, two numbers with the same parity have an **even difference**:

```
Even - Even = Even
Odd  - Odd  = Even
```

So:

same parity $\Rightarrow$ difference is even

This is actually the same remainder idea with:

$$m = 2$$

8. Same remainder modulo 2, 3, 4, etc.

Parity is just a special case.

For modulo 3:

```
Possible remainders:
0 1 2
```

If we have 4 numbers, two must have the same remainder.

Therefore their difference is divisible by 3.

For example:

```
7 % 3 = 1
10 % 3 = 1
```

Therefore:

$$10 - 7 = 3$$

which is divisible by 3.

For modulo 4:

```
0 1 2 3
```

With 5 numbers, two have the same remainder.

Therefore their difference is divisible by 4.

9. Codeforces 1500A — Going Home

Now connect this to the problem you were studying.

Instead of numbers themselves, our **objects are pairs**:

```
(i, j)
```

Each pair gets categorized by:

$$a_i + a_j$$

So:

Pigeons:
pairs of indices
Pigeonholes:
possible pair sums

Because:

$$a_i \leq 2.5 \times 10^6$$

we have:

$$a_i + a_j \leq 5 \times 10^6$$

Therefore there are only about 5 million possible sums.

If we generate more than 5 million pairs:

$$\text{pairs} > \text{possible sums}$$

then two pairs must have the same sum.

So:

$$a_i + a_j = a_k + a_l$$

That's exactly what we want.

10. Duplicate in an array — another famous application

There is a famous problem:

Given $n+1$ integers where every integer is between 1 and n , find a duplicate.

For example:

$n = 4$
[1, 3, 4, 2, 2]

There are:

5 numbers

but only:

4 possible values

Therefore:

$$5 > 4$$

and a duplicate is guaranteed.

This is directly Pigeonhole Principle.

The interesting part is that some problems then ask you to find the duplicate **without extra memory**, leading to algorithms like **Floyd's Cycle Detection**.

So PHP can provide the mathematical guarantee, while another algorithm finds the actual collision efficiently.

11. General competitive-programming pattern

This is the most important thing to learn.

Whenever you see a problem, ask:

Question 1

What are my objects?

Could be:

```
numbers
pairs
prefix sums
indices
strings
people
coordinates
subarrays
```

Question 2

Can I put each object into a category?

For example:

```
number → remainder
pair → sum
person → birthday
number → parity
string → hash
prefix sum → remainder
```

Question 3

How many categories are possible?

For example:

```
mod k → k categories
parity → 2 categories
birthday → 365 categories
pair sum → 5,000,000 categories
```

Question 4

Compare:

number of objects

with:

number of categories

If:

objects > categories

then:

collision is guaranteed

The patterns worth memorizing

Objects	Category	Collision gives
Numbers	Value	Duplicate
Numbers	$a[i] \% k$	Difference divisible by k
Prefix sums	$prefix \% k$	Subarray sum divisible by k
Numbers	Parity	Even difference
People	Birthday	Same birthday
Pairs	Pair sum	Equal pair sums
Numbers	Last digit	Same last digit
Numbers	$a[i] \% m$	Difference is multiple of m

The **big mental connection** is:

Same category

often gives you a useful mathematical relationship.

For example:

same remainder

 $\Rightarrow$

difference divisible by k

and:

$$\boxed{\text{same prefix remainder}} \Rightarrow \boxed{\text{subarray sum divisible by } k}$$

and:

$$\boxed{\text{same pair sum}} \Rightarrow \boxed{a_i + a_j = a_k + a_l}$$

Once you start seeing "**limited possible states/categories**" in a Codeforces problem, you should immediately think: **Pigeonhole Principle**.
 ChatGPT can make mistakes. Check important info.